



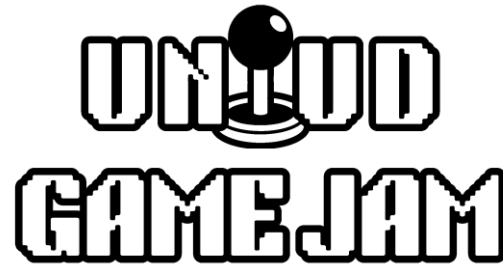


UNIMOD GAMEJAM

UNLUDO
GAME JAM

The logo for Unludo Game Jam features the text "UNLUDO" on the top line and "GAME JAM" on the bottom line. Both lines are rendered in a pixelated, blocky font with a white outline and a dark green fill. The word "UNLUDO" is slightly wider than "GAME JAM". A white pin icon, consisting of a circular head with a small black dot and a cylindrical base, is positioned centrally between the two lines of text, appearing to push the top line down.

Benvenuti alla



Obiettivo: imparare a creare un videogioco da zero



Lavorerete in team per sviluppare un videogioco collaborando con altri partecipanti



Scoprirete e imparerete qualcosa di nuovo, sperimentando tecniche e strumenti del game development

Pronti a iniziare?  

Prima di iniziare...



Vi chiediamo di compilare alcuni questionari iniziali



I dati saranno anonimi (usate il codice utente che vi abbiamo dato)



5-10 minuti





Che cos'è una Game Jam?

Una Game Jam è un evento in cui si crea un videogioco da zero in un tempo limitato, spesso lavorando in team



Tempo limitato: 24-72 ore



Creatività: Nessun limite alle idee, sperimentate e divertitevi



Obiettivo: Creare e pubblicare un gioco entro la fine della Jam

Come verrà gestita la

UNUD
GAMEJAM ?



Come verrà gestita la

UNUO
GAMEJAM ?

Crash Course

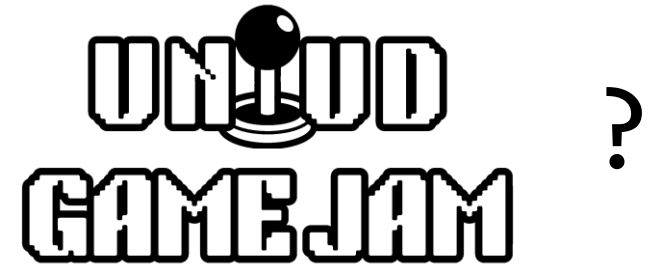
-  Imparerete a **creare** un **videogioco da zero**
-  Le lezioni si svolgeranno nei **laboratori**
-  Imparerete **Unity**, game design e sviluppo collaborativo

Corso intensivo

Non serve essere
programmatori

Trasformare un' idea
in un videogioco

Come verrà gestita la



Ludum Dare è uno degli eventi di Game Jam più famosi al mondo

Creeremo delle squadre bilanciate

I giochi verranno valutati da noi alla fine dell'evento per darvi un riscontro immediato

La Ludum Dare vi permetterà di confrontarvi con sviluppatori di tutto il mondo



Game Jam



Avrete un **tempo limitato** nei laboratori per sviluppare il gioco



Siete liberi di continuare il lavoro anche fuori dagli orari ufficiali



Obiettivo finale: Pubblicare il vostro gioco sulla piattaforma della competizione internazionale Ludum Dare

Struttura dell'evento

Crash Course

Giorno	Orario	Argomento
20/03	14.00-18.00	Tool utili, Game design, basi di Unity
27/03	14.00-18.00	Fisica, Logica del gioco, Asset, Game management
09/04	14.00-18.00	UI, Audio, NavMesh, Animazioni
10/04	14.00-18.00	Rifiniamo il nostro gioco e buildiamolo...

Game Jam

Giorno	Orario	Argomento
17/04	15.00-17.00	Talk @nickzangus, Matchmaking
18/04	9.00-18.00	GAME JAM
19/04	14.00-18.00	GAME JAM
20/04	9.00-16.00 16.00-18.00	GAME JAM e caricamento su Ludum Dare Valutazione, Feedback, Saluti

Supervisori

Biagio



Christian



Massimiliano



Alessandro



-  Durante la fase di formazione, vi guideremo passo dopo passo per imparare a creare un videogioco - ricordate che non serve saper programmare
-  Durante la Game Jam saremo presenti per aiutarvi, rispondere alle vostre domande e supportarvi nello sviluppo del vostro gioco
-  Siamo qui per aiutarvi a trasformare la vostra idea in un videogioco

Domande?



Non ci sono domande sbagliate



Chiedete liberamente, siamo qui per aiutarvi



Il confronto aiuta tutti a imparare di più

Strumenti

- ✓ account [Ludum Dare](#) - piattaforma su cui pubblicare i giochi
- ✓ account [itch.io](#) - piattaforma per condividere e distribuire i giochi
- ✓ GitHub Desktop e account [GitHub](#) - gestione/condivisione del codice
- ✓ Unity 6000.0.70 e un account [Unity](#) - motore di gioco

Li creeremo insieme 🛠️

PC Lab:
i dati vengono
cancellati ogni volta

PC Portatili:
procurarne uno per
team per la Game Jam

Strumenti

- ✓ pacchetti con alcuni asset utili



PC Lab:
i dati vengono
cancellati ogni volta

PC Portatili:
procurarne uno per
team per la Game Jam





Programma della lezione

- 📌 Creiamo un progetto Unity
- 📌 Organizziamo il progetto Unity
- 📌 GameObjects
- 📌 Facciamo muovere i Gameobjects
- 📌 Creiamo dei Prefab
- 📌 Spawniamo ciambelle
- 📌 Miglioriamo la visuale

~~14.00-15.00 introduzione~~
15.00-16.00 basi di Unity
15.30-16.30 GameObjects
16.30-17.30 Ciambelle
17.30-18.00 miglioramenti

Creiamo un progetto Unity



Unity Hub → New Project

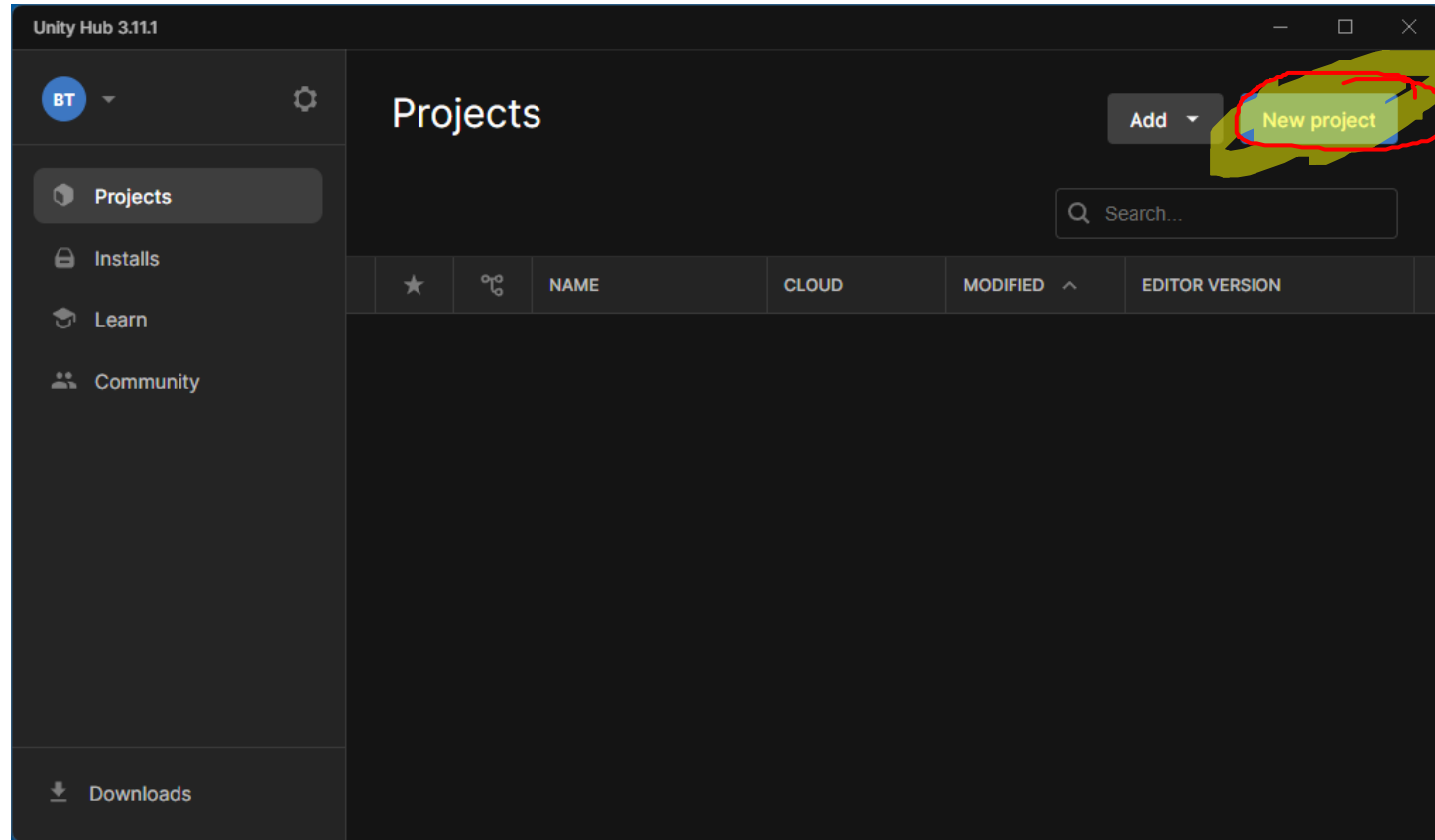


Nome del progetto e selezione template

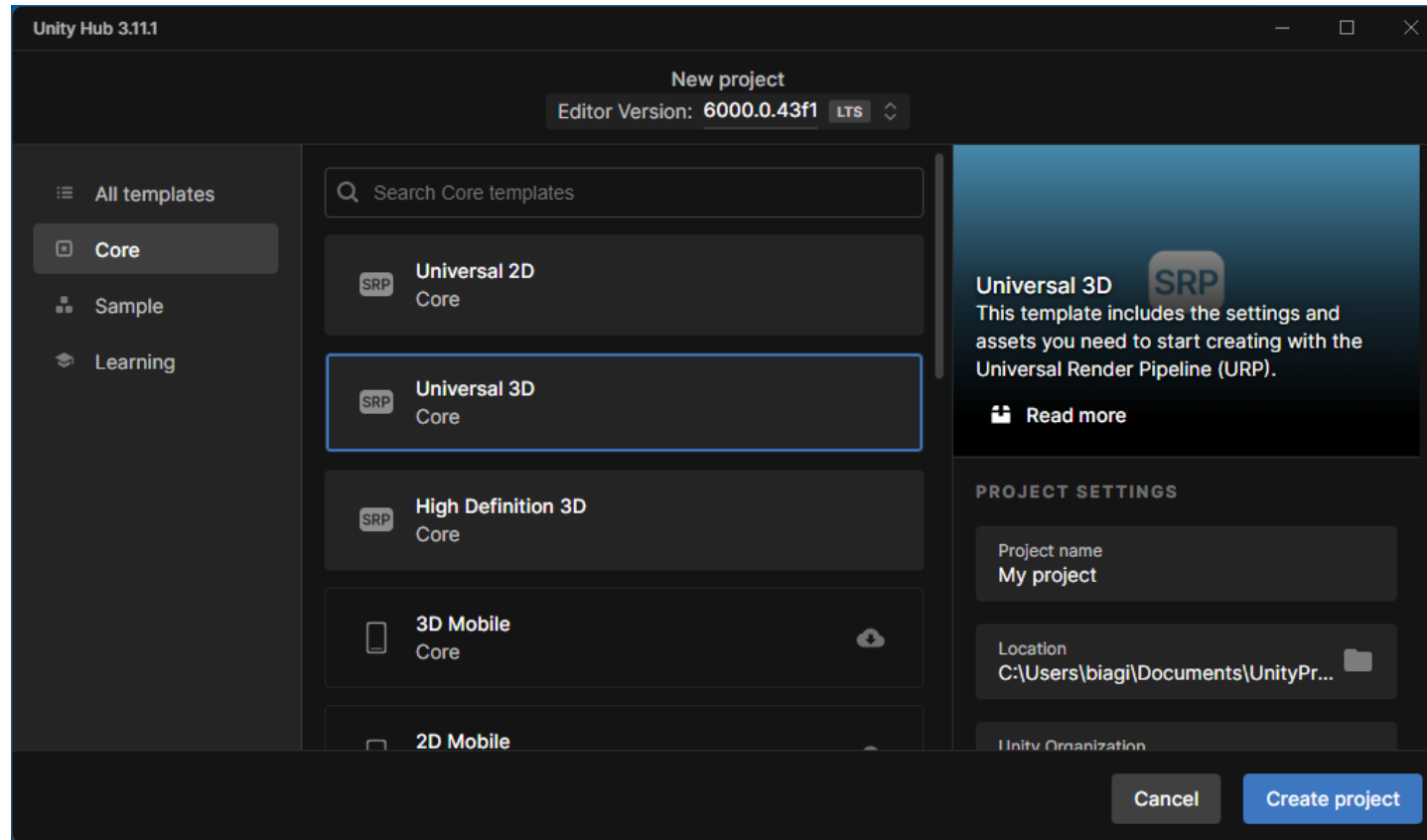


Caricamento del progetto

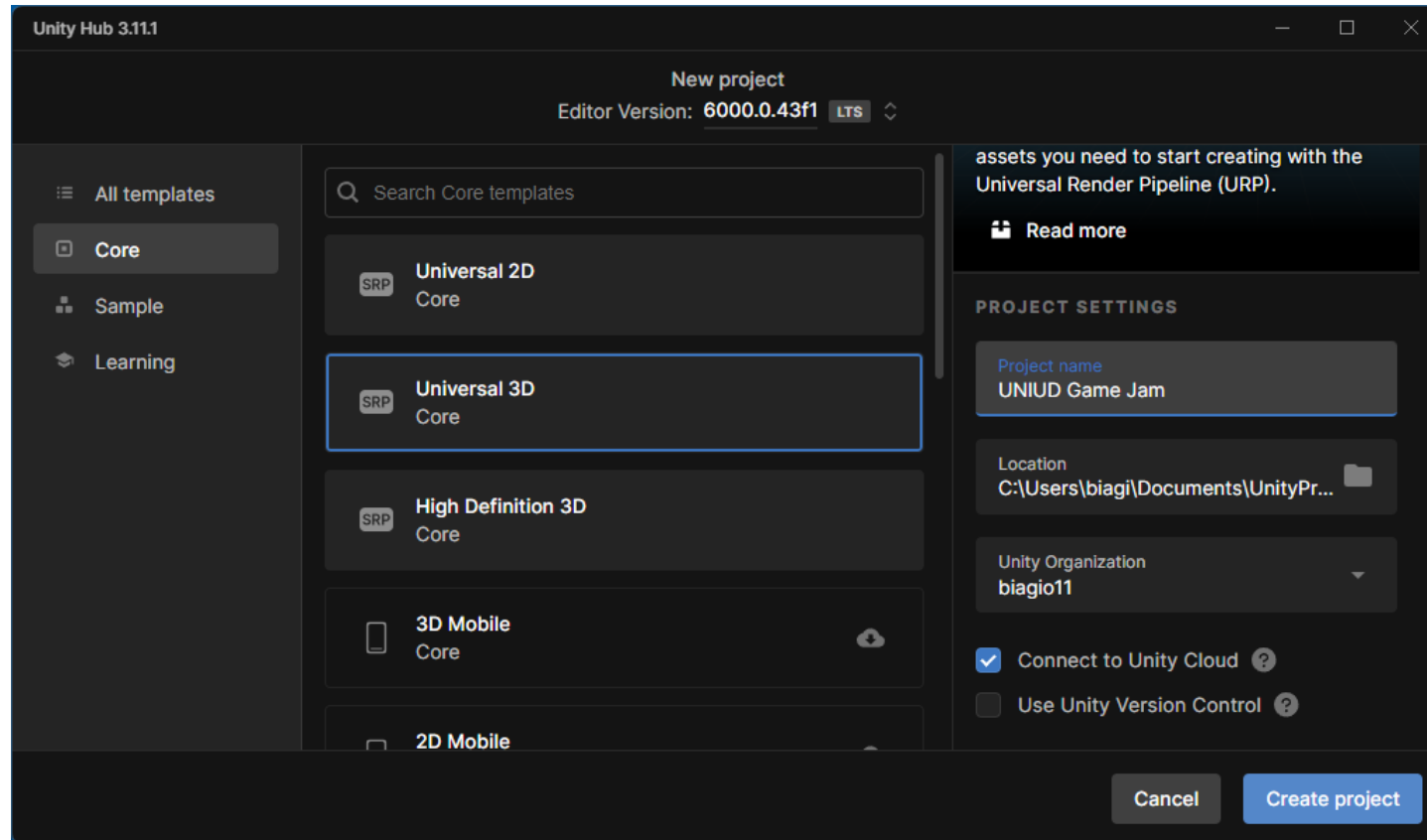
Creiamo un progetto Unity



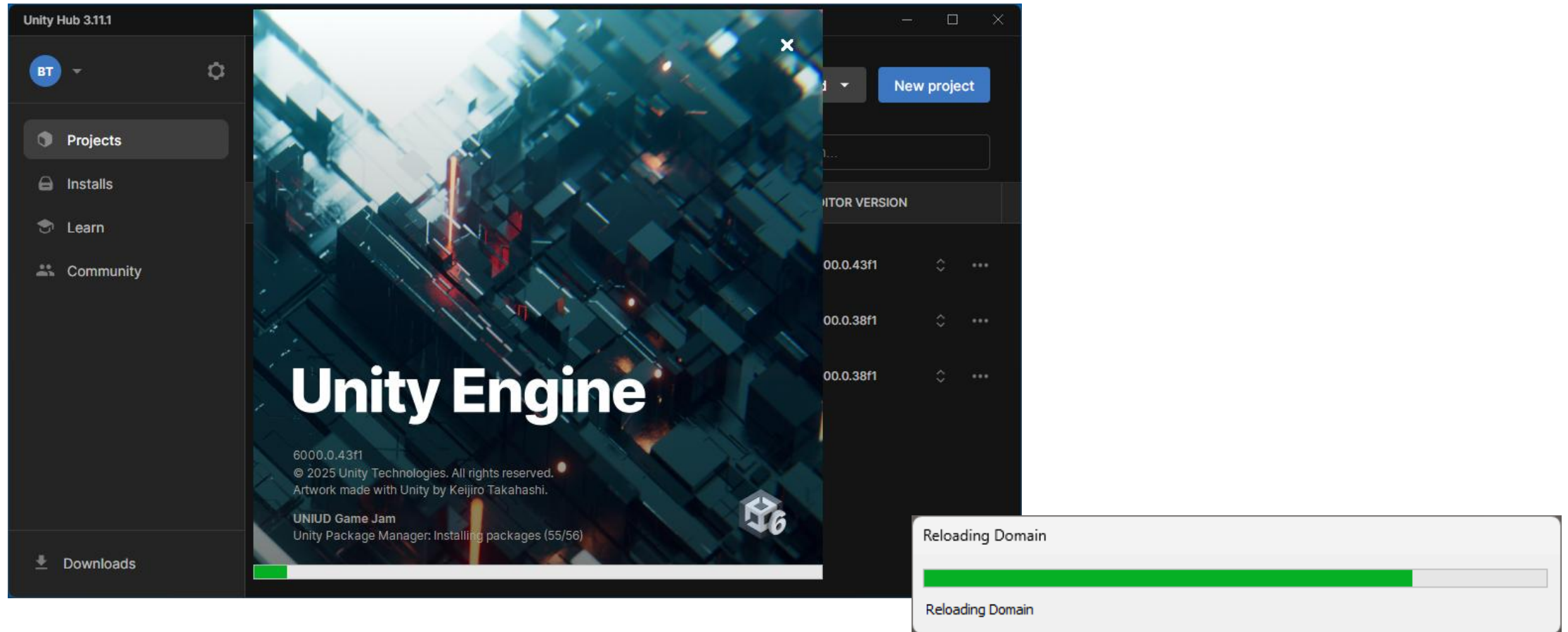
Creiamo un progetto Unity



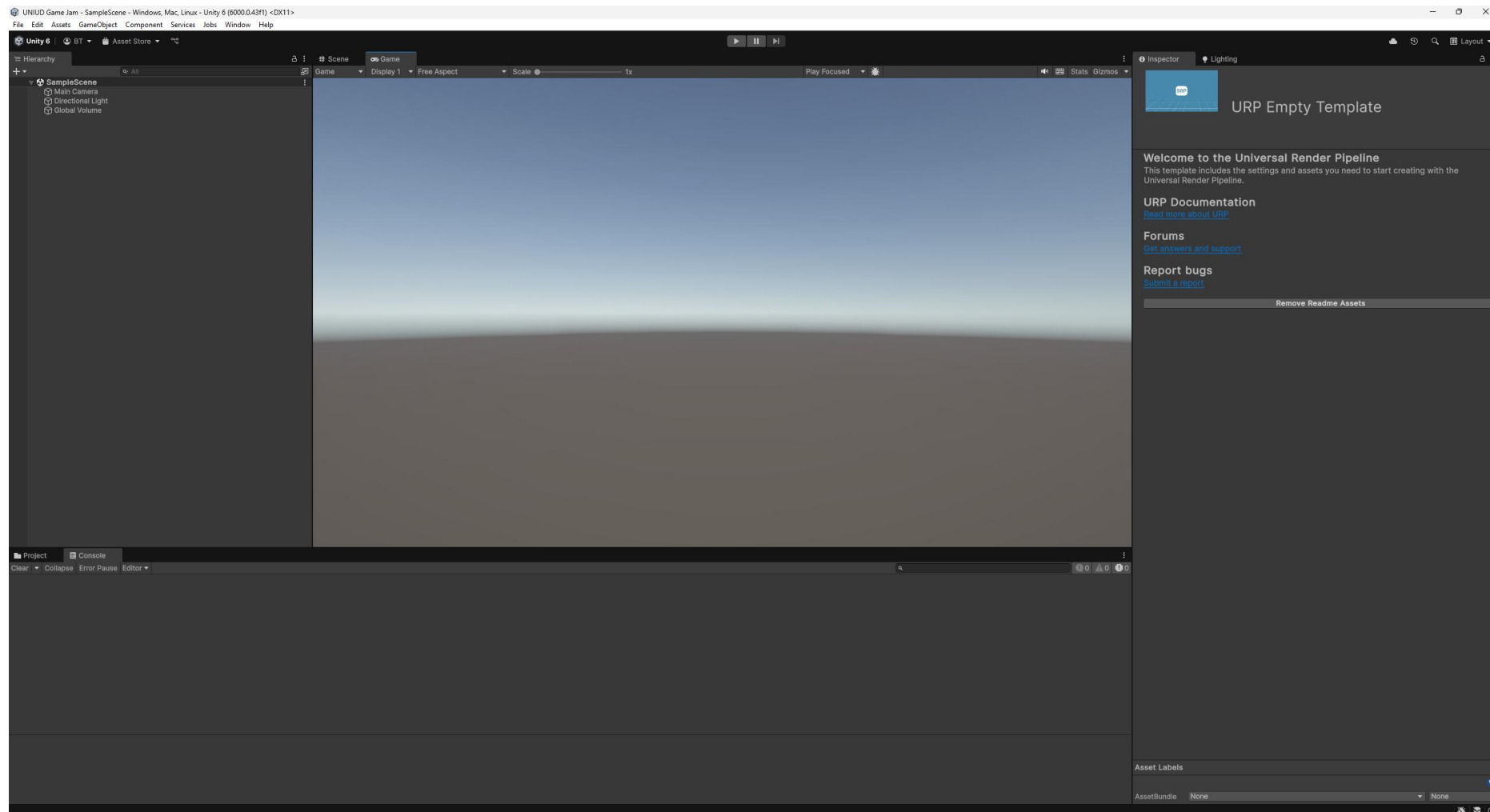
Creiamo un progetto Unity



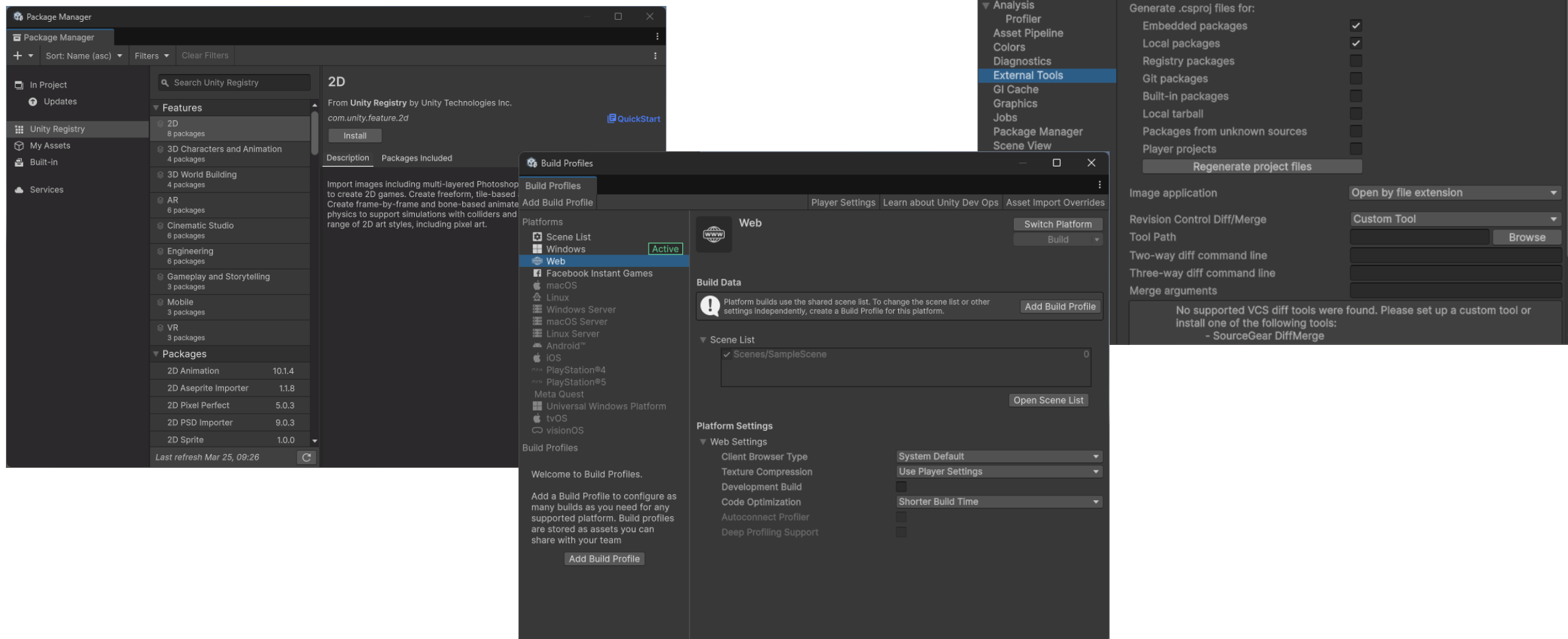
Caricamento...



Unity Editor



Tab importanti...



Organizziamo il progetto Unity

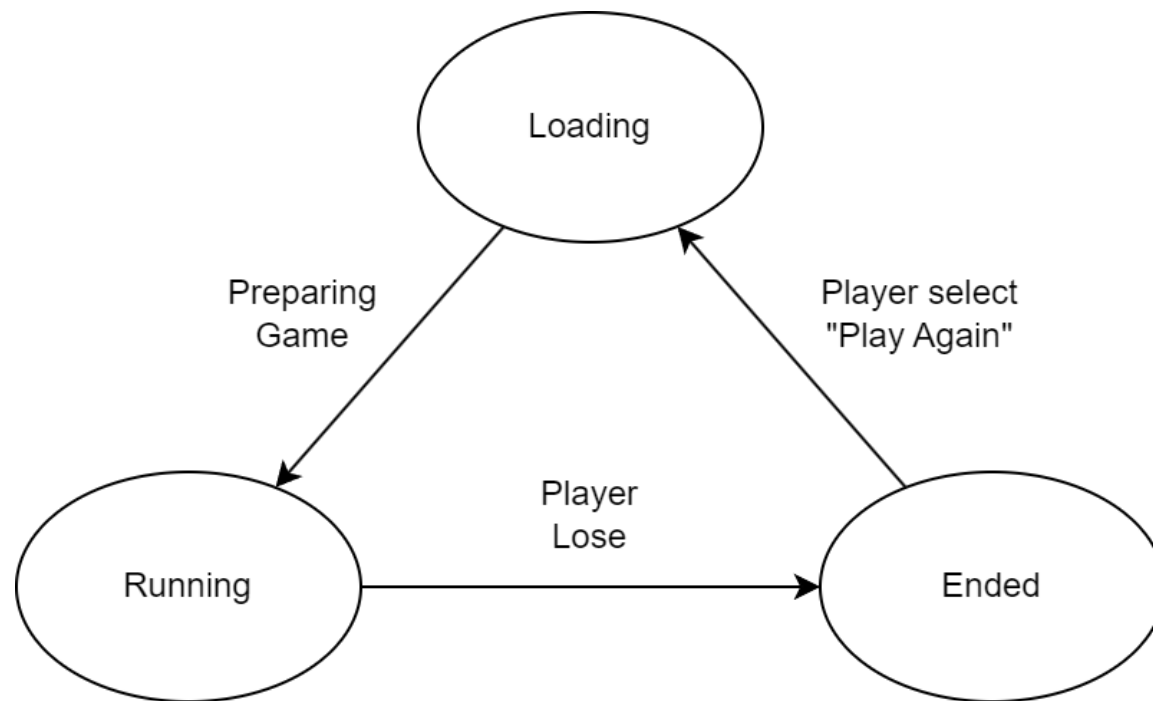
 Creare cartelle per: Art, Materials, Scenes, Scripts, Settings

 Package Manager e Build Profiles

 Edit → Preferences → External Tools

 Game Tab Unfocused

Game logic



Funzioni di base

 Awake

 Start

 Update

```
1 using UnityEngine;
2
3 public class CallCount : MonoBehaviour
4 {
5     // Variable to track total method calls
6     private int methodCallCount = 0;
7
8     // Awake is called when the script instance is being loaded
9     // Use Awake for initial setup and initialization
10    private void Awake()
11    {
12        Debug.Log("Awake");
13    }
14
15    // Start is called before the first frame update
16    // Use Start to prepare game state and components
17    void Start()
18    {
19        Debug.Log("Start");
20    }
21
22    // Update is called once per frame
23    // Use Update for game logic and input handling
24    void Update()
25    {
26        methodCallCount++;
27
28        // Only log periodically to prevent console spam
29        if (methodCallCount % 100 == 0)
30        {
31            Debug.Log($"Update Call #{methodCallCount}");
32        }
33    }
34 }
```

Awake

```
8      // Awake is called when the script instance is being loaded
9      // Use Awake for initial setup and initialization
10     private void Awake()
11     {
12         Debug.Log("Awake");
13     }
```

Start

```
15    // Start is called before the first frame update
16    // Use Start to prepare game state and components
17    void Start()
18    {
19        Debug.Log("Start");
20    }
```


Update

```
22     // Update is called once per frame
23     // Use Update for game logic and input handling
24     void Update()
25     {
26         methodCallCount++;
27
28         // Only log periodically to prevent console spam
29         if (methodCallCount % 100 == 0)
30         {
31             Debug.Log($"Update Call #{methodCallCount}");
32         }
33     }
```




Gameobjects

 Plane (Scala = 10)

 Cube (Y = 0.5)

 Materiali

 Conversione Materiali

Window → Rendering → Initialize and Convert

Facciamoli muovere

✖ Rimuovere Collider

📄 Script Move Object Disattivato

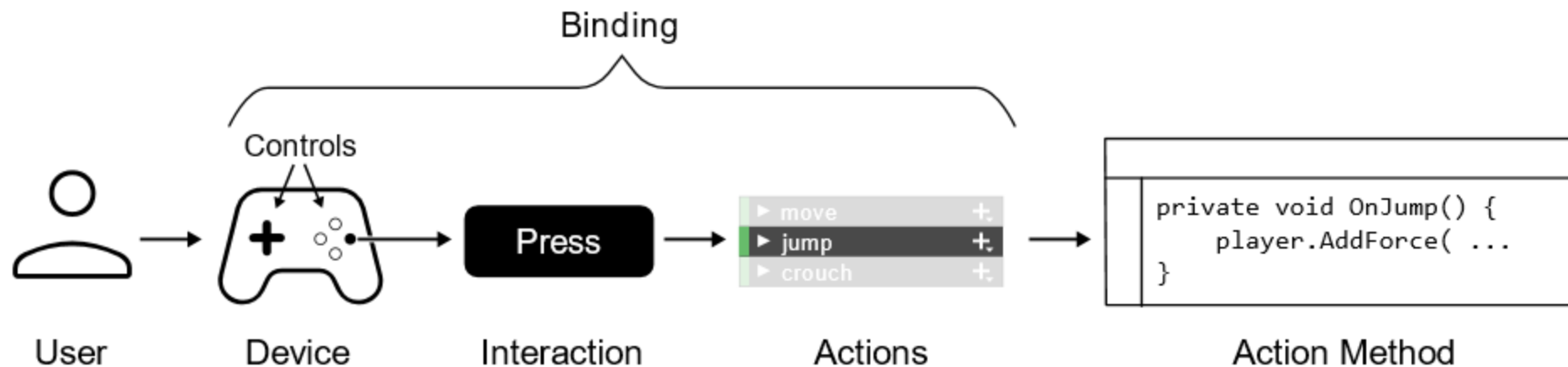
🎮 Player Input

📄 Script Move Character

Move Object

```
1  using UnityEngine;
2
3  public class MoveObject : MonoBehaviour
4  {
5      public Vector3 moveDirection = Vector3.forward;
6      public float moveSpeed = 5f;
7
8      void Update()
9      {
10         // Move the object in the specified direction at the defined speed (meters per second)
11         transform.position += moveDirection * moveSpeed * Time.deltaTime;
12     }
13 }
14
```

Gestiamo l'input



Move Character

```
1 using UnityEngine;
2 using UnityEngine.InputSystem;
3
4 public class MoveCharacter : MonoBehaviour
5 {
6     public float moveSpeed = 5f;
7     public float rotationSpeed = 10f;
8     private Vector2 moveInput;
9     private Vector3 velocity = Vector3.zero;
10
11     public void OnMove(InputValue value)
12     {
13         // Read the 2D movement input
14         moveInput = value.Get<Vector2>();
15         // Debug
16         Debug.Log($"Move input: {moveInput}");
17     }
18
19     void Update()
20     {
21         // Simple movement
22         Vector3 movement = new Vector3(moveInput.x, 0, moveInput.y) * moveSpeed;
23         transform.position += movement * Time.deltaTime;
24
25         // Simple rotation
26         if (movement != Vector3.zero)
27         {
28             transform.rotation = Quaternion.LookRotation(movement, Vector3.up);
29         }
30     }
31 }
```

Move Character

```
11     public void OnMove(InputValue value)
12     {
13         // Read the 2D movement input
14         moveInput = value.Get<Vector2>();
15         // Debug
16         Debug.Log($"Move input: {moveInput}");
17     }
```

Move Character

```
19     void Update()
20     {
21         // Simple movement
22         Vector3 movement = new Vector3(moveInput.x, 0, moveInput.y) * moveSpeed;
23         transform.position += movement * Time.deltaTime;
24
25         // Simple rotation
26         if (movement != Vector3.zero)
27         {
28             transform.rotation = Quaternion.LookRotation(movement, Vector3.up);
29         }
30     }
```

Move Character Migliorato

```
1 using UnityEngine;
2 using UnityEngine.InputSystem;
3
4 public class MoveCharacter : MonoBehaviour
5 {
6     public float moveSpeed = 5f;
7     public float rotationSpeed = 10f;
8     private Vector2 moveInput;
9     private Vector3 velocity = Vector3.zero;
10
11     public void OnMove(InputValue value)
12     {
13         // Read the 2D movement input
14         moveInput = value.Get<Vector2>();
15         // Debug
16         Debug.Log($"Move input: {moveInput}");
17     }
18
19     void Update()
20     {
21         // Smooth movement
22         Vector3 targetVelocity = new Vector3(moveInput.x, 0, moveInput.y) * moveSpeed;
23         velocity = Vector3.Lerp(velocity, targetVelocity, rotationSpeed * Time.deltaTime);
24         transform.position += velocity * Time.deltaTime;
25
26         // Smooth rotation
27         if (velocity != Vector3.zero)
28         {
29             Quaternion toRotation = Quaternion.LookRotation(velocity, Vector3.up);
30             transform.rotation = Quaternion.Slerp(transform.rotation, toRotation, rotationSpeed * Time.deltaTime);
31         }
32     }
33 }
```

Move Character Migliorato

```
19 void Update()
20 {
21     // Smooth movement
22     Vector3 targetVelocity = new Vector3(moveInput.x, 0, moveInput.y) * moveSpeed;
23     velocity = Vector3.Lerp(velocity, targetVelocity, rotationSpeed * Time.deltaTime);
24     transform.position += velocity * Time.deltaTime;
25
26     // Smooth rotation
27     if (velocity != Vector3.zero)
28     {
29         Quaternion toRotation = Quaternion.LookRotation(velocity, Vector3.up);
30         transform.rotation = Quaternion.Slerp(transform.rotation, toRotation, rotationSpeed * Time.deltaTime);
31     }
32 }
```

Move Character Migliorato Ancora

```
4 public class MoveCharacterEnhanced : MonoBehaviour
5 {
6     public float moveSpeed = 5f;
7     public float rotationSpeed = 10f;
8     public Camera cam;
9     private Vector2 moveInput;
10    private Vector3 velocity = Vector3.zero;
11
12    // riferimenti
13    public void OnMove(InputValue value)
14    {
15        moveInput = value.Get<Vector2>();
16        //Debug.Log($"Move input: {moveInput}");
17    }
```


Move Character Migliorato Ancora

```
18 void Update()
19 {
20     Vector3 camForward = Vector3.ProjectOnPlane(cam.transform.forward, Vector3.up).normalized;
21     Vector3 camRight = Vector3.ProjectOnPlane(cam.transform.right, Vector3.up).normalized;
22     Vector3 inputDir = (camRight * moveInput.x + camForward * moveInput.y);
23
24     // Smooth movement
25     Vector3 targetVelocity = inputDir * moveSpeed;
26     velocity = Vector3.Lerp(velocity, targetVelocity, rotationSpeed * Time.deltaTime);
27     transform.position += velocity * Time.deltaTime;
28
29     // Smooth rotation - don't rotate when moving backward
30     if (inputDir.sqrMagnitude > 0.01f && moveInput.y >= 0f)
31     {
32         Quaternion toRotation = Quaternion.LookRotation(inputDir, Vector3.up);
33         transform.rotation = Quaternion.Slerp(transform.rotation, toRotation, rotationSpeed * Time.deltaTime);
34     }
35 }
36 }
```

Move Routine

```
5  ✓ public class MoveObjectRoutine : MonoBehaviour
6  {
7      [System.Serializable]
8      ✓ public class MoveStep
9      {
10         public Vector3 direction = Vector3.forward;
11         public float distance = 2f;
12         public float speed = 5f;
13         public float waitAfter = 0f; // optional pause after this step
14     }
15
16     public List<MoveStep> steps = new List<MoveStep>();
17     public bool loop = false;
18     public bool playOnStart = true;
19
20     private Coroutine _activeRoutine;
21
```

Move Routine

```
45 private IEnumerator RunSequence()
46 {
47     do
48     {
49         foreach (var step in steps)
50         {
51             Vector3 target = transform.position + step.direction.normalized * step.distance;
52
53             while (Vector3.Distance(transform.position, target) > 0.01f)
54             {
55                 transform.position = Vector3.MoveTowards(transform.position, target, step.speed * Time.deltaTime);
56                 yield return null;
57             }
58
59             transform.position = target;
60
61             if (step.waitAfter > 0f)
62                 yield return new WaitForSeconds(step.waitAfter);
63         }
64     } while (loop);
65
66     _activeRoutine = null;
67 }
68
```

Move Routine

```
22  void Start()
23  {
24      if (playOnStart)
25          Play();
26  }
27
28  1 riferimento
29  public void Play()
30  {
31      if (_activeRoutine != null)
32          StopCoroutine(_activeRoutine);
33
34      _activeRoutine = StartCoroutine(RunSequence());
35  }
36
37  0 riferimenti
38  public void Stop()
39  {
40      if (_activeRoutine != null)
41      {
42          StopCoroutine(_activeRoutine);
43          _activeRoutine = null;
44      }
45  }
```

Creiamo dei Prefab

 Creare un Prefab

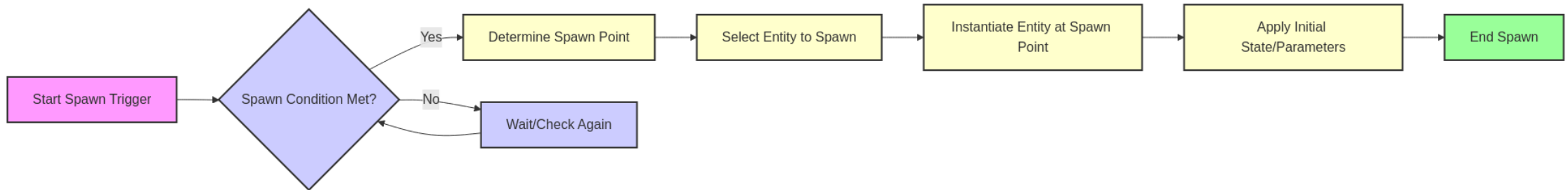
 New Parent

 Unpack

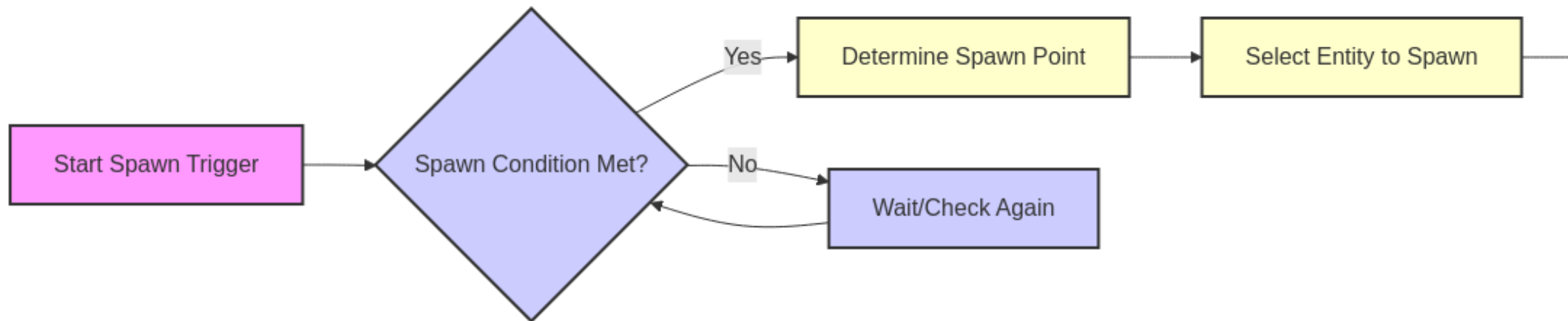
 Prefab Variant



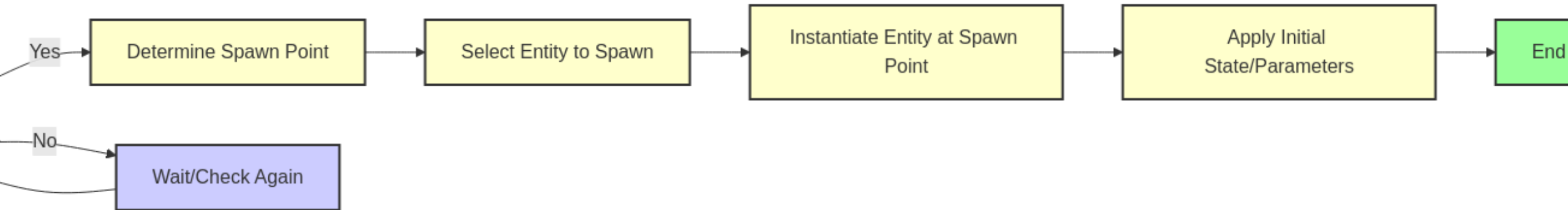
Logica di spawn



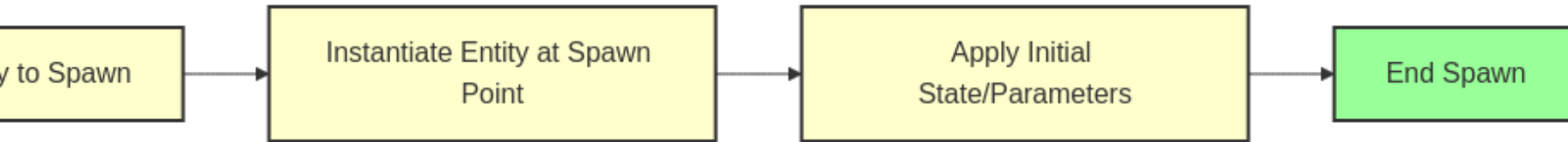
Logica di spawn



Logica di spawn



Logica di spawn



Spawniamo ciambelle

 Script per spawnare ciambelle

 Animazioni con scala per effetti visivi

Spawniamo ciambelle

```
1 public class RandomObjectSpawner : MonoBehaviour
2 {
3     [Header("Spawn Settings")]
4     [Tooltip("Prefab to spawn")]
5     public GameObject objectToSpawn;
6
7     [Tooltip("Maximum number of objects to spawn")]
8     [Range(1, 100)]
9     public int maxSpawnCount = 10;
10
11     ...
```


Spawniamo ciambelle

```
1 private void SetNextSpawnTime()
2     {
3         // Set the next spawn time with some randomness
4         nextSpawnTime = Time.time + Random.Range(minSpawnInterval, maxSpawnInterval);
5     }
```

```
1 public void ResetSpawner()
2     {
3         currentSpawnCount = 0;
4         SetNextSpawnTime();
5     }
```

Spawniamo ciambelle

```
1 private void SpawnObject()
2 {
3     // Get the renderer of the spawn plane to determine its bounds
4     Renderer planeRenderer = spawnPlane.GetComponent<Renderer>();
5
6     if (planeRenderer == null)
7     {
8         Debug.LogError("Spawn plane must have a Renderer component!");
9         return;
10    }
11
12    // Calculate random position within the plane's bounds
13    float randomX = Random.Range(planeRenderer.bounds.min.x, planeRenderer.bounds.max.x);
14    float randomZ = Random.Range(planeRenderer.bounds.min.z, planeRenderer.bounds.max.z);
15
16    // Use the plane's Y position plus a small offset to prevent clipping
17    Vector3 spawnPosition = new Vector3(
18        randomX,                // x
19        spawnPlane.transform.position.y + 0.1f, // y
20        randomZ                 // z
21    );
22
23    // Instantiate the object as a child of this GameObject
24    GameObject spawnedObject = Instantiate(objectToSpawn, spawnPosition, Quaternion.identity, transform);
25
26    // Apply fixed scale
27    spawnedObject.transform.localScale = Vector3.one * spawnScale;
28
29    // Increment spawn count
30    currentSpawnCount++;
31 }
```

Spawniamo ciambelle

```
1 private void SpawnObject()
2 {
3     // Get the renderer of the spawn plane to determine its bounds
4     Renderer planeRenderer = spawnPlane.GetComponent<Renderer>();
5
6     if (planeRenderer == null)
7     {
8         Debug.LogError("Spawn plane must have a Renderer component!");
9         return;
10    }
11
12    // Calculate random position within the plane's bounds
13    float randomX = Random.Range(planeRenderer.bounds.min.x, planeRenderer.bounds.max.x);
14    float randomZ = Random.Range(planeRenderer.bounds.min.z, planeRenderer.bounds.max.z);
15
16    // Use the plane's Y position plus a small offset to prevent clipping
17    Vector3 spawnPosition = new Vector3(
18        randomX,                                // x
19        spawnPlane.transform.position.y + 0.1f, // y
20        randomZ                                  // z
21    );
```

Spawniamo ciambelle

```
16 // Use the plane's Y position plus a small offset to prevent clipping
17 Vector3 spawnPosition = new Vector3(
18     randomX, // x
19     spawnPlane.transform.position.y + 0.1f, // y
20     randomZ // z
21 );
22
23 // Instantiate the object as a child of this GameObject
24 GameObject spawnedObject = Instantiate(objectToSpawn, spawnPosition, Quaternion.identity, transform);
25
26 // Apply fixed scale
27 spawnedObject.transform.localScale = Vector3.one * spawnScale;
28
29 // Increment spawn count
30 currentSpawnCount++;
31 }
```

Spawniamo ciambelle

```
1 private void Update()  
2 {  
3     // Check if we can spawn more objects  
4     if (currentSpawnCount < maxSpawnCount && Time.time >= nextSpawnTime)  
5     {  
6         SpawnObject();  
7         SetNextSpawnTime();  
8     }  
9 }
```

Spawning ad effetto

```
1 public class ScaleAnimation : MonoBehaviour
2 {
3     [Tooltip("Duration of the scaling animation")]
4     public float animationDuration = 1f;
5
6     private Vector3 targetScale;
7     private float animationTimer;
8
9     private void Start()
10    {
11        // Store the target scale as the current scale of the object
12        targetScale = transform.localScale;
13
14        // Start with zero scale
15        transform.localScale = Vector3.zero;
16
17        // Start the animation
18        StartAnimation();
19    }
```


Spawning ad effetto

```
1 public void StartAnimation()  
2 {  
3     animationTimer = 0f;  
4     InvokeRepeating(nameof(UpdateScale), 0f, Time.deltaTime);  
5 }
```

```
1 public void ResetAnimation()  
2 {  
3     transform.localScale = Vector3.zero;  
4 }
```

Spawning ad effetto

```
1 private void UpdateScale()
2     {
3         // Increment timer
4         animationTimer += Time.deltaTime;
5
6         // Calculate normalized time (0 to 1)
7         float t = Mathf.Clamp01(animationTimer / animationDuration);
8
9         // Linearly interpolate the scale from zero to target scale
10        transform.localScale = Vector3.Lerp(Vector3.zero, targetScale, t);
11
12        // Stop animation when complete
13        if (t >= 1f)
14        {
15            CancelInvoke(nameof(UpdateScale));
16        }
17    }
```




Miglioriamo la visuale



Miglioriamo la visuale



Script Camera in terza persona



Inseriamo la nuova input action (C, come Camera switch)



Skybox



Ottimizzazioni Camera

Miglioriamo la visuale

[root/scripts](#)
[CameraController.cs](#)

```
1 public class CameraController : MonoBehaviour
2 {
3     // Camera modes
4     public enum CameraMode
5     {
6         ThirdPerson,
7         TopDown
8     }
```

Miglioriamo la visuale

```
1 // Camera configuration
2 public CameraMode currentMode = CameraMode.ThirdPerson;
3 public Transform target; // Character to follow
4
5 // Third-person camera settings
6 public Vector3 thirdPersonOffset = new Vector3(0, 8, -5);
7 public float thirdPersonAngle = 50f;
8
9 // Top-down camera settings
10 public Vector3 topDownOffset = new Vector3(0, 10, -8);
11 public float topDownAngle = 45f;
12
13 // Camera component
14 private Camera mainCamera;
```

Miglioriamo la visuale

```
1 void Start()
2 {
3     // If no target is assigned, try to use the parent object
4     if (target == null)
5         target = transform;
6
7     mainCamera = Camera.main;
8     if (mainCamera == null)
9     {
10         Debug.LogError("No main camera found in the scene!");
11         enabled = false;
12     }
13 }
```


Miglioriamo la visuale

```
1 // LateUpdate is called after Update each frame
2 void LateUpdate()
3 {
4     // Position the camera based on the current mode
5     switch (currentMode)
6     {
7         case CameraMode.ThirdPerson:
8             UpdateThirdPersonCamera();
9             break;
10        case CameraMode.TopDown:
11            UpdateTopDownCamera();
12            break;
13    }
14 }
```

Miglioriamo la visuale

```
1 void UpdateThirdPersonCamera()
2 {
3     // Calculate camera position with offset
4     Vector3 desiredPosition = target.position + target.TransformDirection(thirdPersonOffset);
5     mainCamera.transform.position = desiredPosition;
6
7     // Calculate camera rotation
8     Quaternion desiredRotation = Quaternion.Euler(thirdPersonAngle, target.eulerAngles.y, 0);
9     mainCamera.transform.rotation = desiredRotation;
10 }
```

Miglioriamo la visuale

```
1 void UpdateTopDownCamera()
2 {
3     // Position camera directly above the target
4     Vector3 desiredPosition = target.position + topDownOffset;
5     mainCamera.transform.position = desiredPosition;
6
7     // Look straight down
8     mainCamera.transform.rotation = Quaternion.Euler(topDownAngle, 0, 0);
9 }
```

Miglioriamo la visuale

```
1 // Input System method to switch camera modes
2 public void OnCameraSwitch(InputValue value)
3 {
4     // Cycle between available modes
5     currentMode = (currentMode == CameraMode.ThirdPerson)
6         ? CameraMode.TopDown
7         : CameraMode.ThirdPerson;
8
9     // Optional: Log the current camera mode
10    Debug.Log($"Switched to {currentMode} Camera Mode");
11 }
```



